

Ginja AI Document Extraction Pipeline:

WRITE-UP

Candidate: Timothy Amwata **Role:** AI/ML Engineering Intern

1. Introduction of the Problem

Healthcare claims in emerging markets suffer from extreme format variability, ranging from cleanly typed digital PDFs to low-quality, handwritten, and stamped paper scans. I initially approached this task as a standard data engineering pipeline challenge. However, as I progressed, I realized it is fundamentally an ML architecture challenge—balancing the deterministic speed of traditional OCR against the semantic reasoning of modern Vision Language Models (VLMs).

Assuming the input documents are already staged by upstream data engineering pipelines, my solution focuses on building a robust, fault-tolerant extraction engine. The overarching goal is not just perfect extraction, but observable data validation—ensuring that missing or malformed fields (critical for fraud detection) are flagged rather than silently dropped.

2. System Flowchart

The system implements a sequential pipeline that dynamically routes documents based on their underlying data structure before normalizing and validating the output.

[START: Document Ingestion]

|
v

[1. FORMAT DETECTION] -> Check for embedded text layer (.pdf vs image)

|
|--

IF TEXT PDF: -> [2a. Extract via pdfplumber]

|
|--

IF SCANNED/IMAGE: -> [2b. Image Preprocessing (cv2)] -> [2c. Tesseract OCR]

[3. DATA PARSING] -> Execute RegEx to extract raw fields from unstructured text.

|
|

[4. NORMALIZATION] -> Convert Dates (ISO 8601), Floats (Amounts), & Uppercase Codes.

|
|

[5. VALIDATION] -> Check completeness, date plausibility, and amount ranges.

```
|      -> Append `validation_flags` and `fields_missing`.
|
[6. EXPORT] -> Assemble unified Schema -> Output to structured JSON and CSV.
```

3. Pseudocode Structure

To illustrate the programmatic logic of the master pipeline, the execution sequence is summarized in the pseudocode below:

```
FUNCTION MAIN_PIPELINE():
    master_records = []

    FOR EACH document IN document_list:
        format_type = DETECT_FORMAT(document)

        # Routing Phase
        IF format_type == "TEXT_PDF":
            raw_text = EXTRACT_PDF_TEXT(document)
            method = "pdf_text"
        ELSE:
            clean_image = PREPROCESS_FOR_OCR(document)
            raw_text = RUN_TESSERACT_OCR(clean_image)
            method = "ocr"

        # Parsing & Validation Phase
        raw_dict = PARSE_WITH_REGEX(raw_text)
        clean_dict = NORMALIZE_DATA(raw_dict)
        missing_keys, flags = VALIDATE(clean_dict)

        # Assembly
        final_record = {
            "file_name": document.name,
            "extraction_method": method,
            "extracted_data": clean_dict,
            "fields_missing": missing_keys,
            "validation_flags": flags
        }
        APPEND final_record TO master_records

    SAVE_TO_CSV(master_records)
    SAVE_TO_JSON(master_records)
```

(Note: To facilitate direct comparison and avoid rate limits during batch processing, the Groq VLM extraction was built as an independent parallel module rather than being nested inside this primary loop).

4. Noticeable Observations & Challenges Encountered

A. Traditional OCR vs. VLM: Brittleness vs. Semantics A core question the prompt poses is why we should compare OCR with VLMs if they target similar outcomes. During execution, the answer became obvious: *semantics vs. rigid rules*. My traditional OCR pipeline (`pytesseract`) successfully pulled text, but extracting the target fields required Regular Expressions (e.g., searching for "`Member No:`"). This is highly brittle; if a hospital changes its invoice template to read "`Patient ID:`", the RegEx fails, and the pipeline drops the data. In contrast, the Groq VLM utilizes semantic reasoning. When prompted with a JSON schema for "`member_id`", the VLM understands that "Patient ID" is contextually identical. The trade-off is that OCR is extremely fast and computationally cheap, while the VLM introduces API latency and cost, but handles layout variability flawlessly.

B. Groq API Model Churn I encountered a real-world API dependency issue. Groq recently updated their naming conventions, decommissioning `llama-3.2-90b-vision-preview` and removing the word "vision" from new multimodal IDs (e.g., `llama-4-scout` and `qwen3.6-27b`). My initial dynamic fetch function—which searched for models containing the word "vision"—failed. I had to refactor the code to explicitly search for new active multimodal prefixes, demonstrating the necessity of robust error handling when relying on third-party AI endpoints.

Additionally, to ensure security, I designed the pipeline to accept the `Groq_API_KEY` via Colab Secrets rather than hardcoding it into the notebook, ensuring my credentials are not leaked in the GitHub submission.

C. PIL Decompression Bomb Warning When converting high-resolution PDFs to images for OCR at 300 DPI, the Python Imaging Library (PIL) threw a `DecompressionBombWarning`. Because the pixel count exceeded PIL's default safety limit designed to prevent DOS attacks, I had to explicitly override `Image.MAX_IMAGE_PIXELS = None` after validating that our input data source was trusted.

D. The Philosophy of Data Validation A major focus was placed on the validation module. Instead of writing `if` statements that drop records with missing fields or negative amounts, the pipeline preserves the record and tags it with `validation_flags`. In a health-tech context, silent failures are dangerous. Keeping flagged data is critical for two reasons: establishing audit trails for hospital disputes, and building historical databases to detect systemic fraud.

5. What I Would Do With More Time (Streamlit & Architecture)

If given more time to transition this MVP into a production system, I would implement two major additions:

1. **A Cascading Fallback Architecture:** Currently, OCR and VLM run in parallel for comparison. In production, I would cascade them to optimize cost. The system would attempt free, local OCR first. Only if the validation module flags a critical missing field (like `claimed_amount`) would the system programmatically route that specific, difficult document to the Groq VLM fallback.
2. **Streamlit "Human-in-the-Loop" Dashboard:** I would build an interactive Streamlit GUI. While the pipeline automates the easy 90%, the dashboard would serve as an Exception Queue, highlighting records with `validation_flags` so a human operator could visually compare the raw document against the parsed JSON and manually correct OCR errors.

6. Thoughts on Final Outputs

The final pipeline successfully generates both CSV and JSON formats. Crucially, the structure goes beyond just extracting the target schema; it encapsulates the metadata (`extraction_method`, `confidence_score`, `validation_flags`). By maintaining this metadata, the pipeline doesn't just output raw data—it outputs *contextualized* data, which is exactly what upstream decision engines need to automate claims adjudication and flag fraud accurately.

7. Author's Note on Methodology

As a 4th-year electrical engineering student, this was my first time architecting an end-to-end ML data extraction pipeline of this nature. To bridge my knowledge gaps rapidly, I utilized official documentation, engineering blogs, and Large Language Models (LLMs) as interactive research assistants. This methodology allowed me to deeply grasp the underlying architectural principles, API handling, and trade-offs of each extraction method, ensuring I fully understood the *why* behind the code rather than simply executing the *how*.